Lesson 14 — Controller Tuning and Stability

Physical AI | Module 3: Dynamics & Control | Completed

Learning objectives

- Explain the role of proportional and derivative gains in a PD controller.
- Derive natural frequency and damping ratio for the simplified gravity-compensated joint.
- Compare underdamped, near-critically damped, and overdamped responses with Python.
- Identify limitations of this simplified model.

1. Recap: PD control with gravity compensation

For a single joint rotating in a vertical plane, with angle measured from the horizontal:

```
I * theta_ddot + b * theta_dot + m*g*L*cos(theta) = tau
```

Here I is inertia [kg m^2], b is viscous damping [N m s/rad], m is payload mass [kg], L is distance to its center of mass [m], and tau is applied motor torque [N m]. The model treats the load as a point mass at distance L and assumes constant I.

With a fixed target theta_d, the PD + gravity compensation law is:

```
tau = Kp*(theta_d - theta) - Kd*theta_dot + m*g*L*cos(theta)
```

Substitution gives the closed-loop error equation:

```
I * theta_ddot + (b + Kd)*theta_dot + Kp*(theta - theta_d) = 0
```

2. Tuning formulas

```
natural frequency: omega_n = sqrt(Kp / I) [rad/s]

damping ratio:     zeta = (b + Kd) / (2*sqrt(I*Kp))
```

- 0 < zeta < 1: underdamped; may overshoot and oscillate.
- zeta = 1: critically damped in the ideal second-order system.
- zeta > 1: overdamped; no oscillation but typically slower response.

These classifications assume the ideal linearized error equation with constant target and exact gravity compensation. Nonlinearities, actuator saturation, latency, and discrete simulation may affect real responses.

3. Python experiment

Save as lesson_14_controller_tuning.py and run in a Python environment with NumPy and Matplotlib.

```
import numpy as np
import matplotlib.pyplot as plt

I = 0.5
b = 0.2
m, L, g = 1.0, 0.5, 9.81
target = np.deg2rad(45)
dt, duration = 0.005, 3.0
steps = int(duration / dt)
controllers = {
    "Underdamped": (12.0, 0.3),
    "Near-critical": (12.0, 4.7),
    "Overdamped": (12.0, 10.0),
}

def simulate(Kp, Kd):
    theta, omega = 0.0, 0.0
    angles = [0.0]
    for _ in range(steps):
        error = target - theta
        tau = Kp*error - Kd*omega + m*g*L*np.cos(theta)
        alpha = (tau - b*omega - m*g*L*np.cos(theta)) / I
        omega += alpha*dt
        theta += omega*dt
        angles.append(np.rad2deg(theta))
    return np.array(angles)

time = np.linspace(0, duration, steps + 1)
for name, (Kp, Kd) in controllers.items():
    angles = simulate(Kp, Kd)
    zeta = (b + Kd) / (2*np.sqrt(I*Kp))
    overshoot = max(0.0, np.max(angles) - 45.0)
    final_error = abs(45.0 - angles[-1])
    print(name, f'zeta={zeta:.3f}',
          f'overshoot={overshoot:.3f} deg',
          f'final_error={final_error:.3f} deg')
    plt.plot(time, angles, label=name)
plt.axhline(45, ls='--', label='Target')
plt.xlabel('Time (s)')
plt.ylabel('Joint angle (deg)')
plt.title('Lesson 14: PD Controller Tuning')
plt.grid(True)
plt.legend()
plt.tight_layout()
plt.savefig('lesson_14_controller_tuning.png', dpi=160)
plt.show()
```

4. Analytical gain comparison

For I = 0.5, b = 0.2, and Kp = 12, the denominator is 2*sqrt(6) = 4.89898:

```
| Case | Kp | Kd | Damping ratio |

|---|---:|---:|---:|

| Underdamped | 12 | 0.3 | 0.102 |

| Near-critical | 12 | 4.7 | 1.000 |

| Overdamped | 12 | 10.0 | 2.082 |
```

Your reported Python experiment generated the three response curves and matched the expected qualitative behavior. Actual overshoot and final-error values depend on the numerical run.

5. Practical robotics implications

- Kp sets stiffness and, with inertia, natural frequency.
- Kd adds damping; excessive derivative gain can slow response and amplify velocity-sensor noise.
- Real joints also have torque limits, joint limits, friction, delay, and modeling uncertainty.
- Gravity compensation here is exact by assumption; real control must tolerate errors.

6. Assessment record

Python: passed (user-reported).

Question B: Increasing derivative gain generally increases damping and can reduce oscillation. Your answer: B, correct.

Question C: Damping ratio near 1 corresponds to critical damping in this ideal model. Your answer: A, correct.

Result: 3/3 — Lesson 14 completed.

7. Review questions

1. Explain why high proportional gain alone can lead to overshoot.
2. Compute zeta for I=0.5, b=0.2, Kp=12, Kd=4.7.
3. Why does increasing Kd not always improve a real controller?
4. How would actuator torque saturation change the ideal response?

Next lesson: Lesson 15 — Torque limits, saturation, and anti-windup foundations.